Design Procedure

For this lab, we were tasked with designing and implementing a parking lot occupancy counter. The system tracks cars entering and exiting a parking lot using two photosensor inputs and displays the occupancy on seven-segment displays. The seven-segment display also shows if the parking lot is full or clear.

Task #1 Car Detection FSM

Car detection has two input signals: outer and inner, in the form 00. When a car begins entering the lot, the outer signal turns on while the inner signal stays at zero (10). When the car is in the middle of the sensors, both outer and inner signals are on (11). We designed an FSM to keep track of the position of the car that has two outputs whether the car is entering the lot or exiting the lot. The way we designed the FSM in figure 1 below was to output enter or exit after the car has completed its motion through the sensors. In figure 1, when ps == "11", the only way for a car to be in that state is if it has first passed through the "10" or "01" states. This is due to the fact that a car cannot go backwards once it is already in the sensors. So, when ps == "11" and ns == "01" we know that the car is entering the lot and exiting if ns == "10". We also handled the case where a pedestrian is walking through the sensors, sending ns == "00" when it tries to go from "10" to "01" or vice versa (exact module logic on Figure 2).

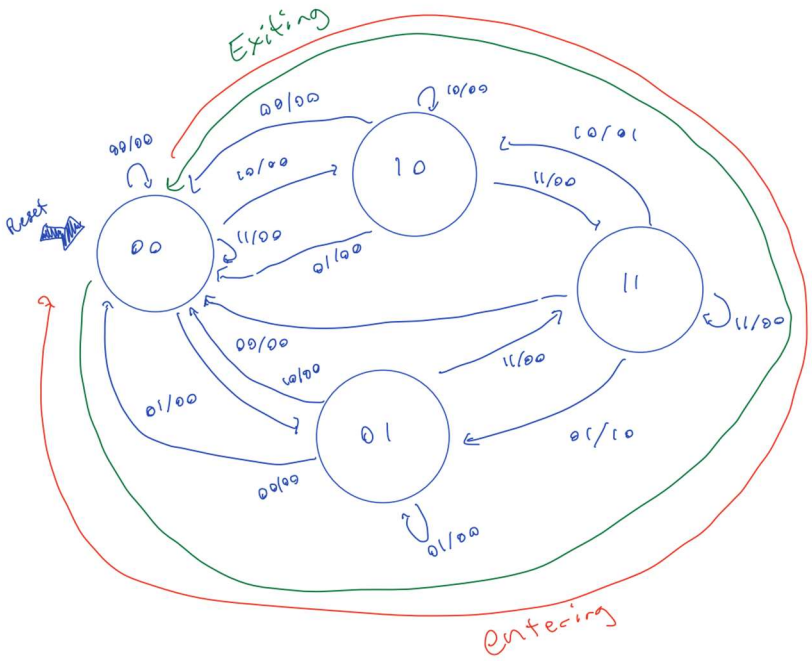

Figure 1: Car Detection FSM that keeps track of whether a car is entering the lot or not.

```
assign enter = (ps == OneOne && ns == ZeroOne); //enter will only be valid during the transition None state to Enter state
assign exit = (ps == OneOne && ns == OneZero); //exit will only be valid during the transition between None state and Exit state
```

Figure 2: Exact assign logic for enter and exit.

Task #2 Car Counter FSM

To keep track of the number of cars in the parking lot we designed and implemented a car counter. This FSM in figure 2 below has one output, keeping track of the number of cars in the lot. When increment is pressed, it will go to the next state and output the number associated with that state. When it reaches 18, it will stay at 18 even when increment is pressed. Similarly, when it reaches zero, it will stay at zero even when decrement is pressed (note total is our output so FULL and CLEAR are out of the scope of this FSM).

Figure 3: FSM for car counter (outputs in red).

Overall System: Top Level Block Diagram

The final task was combining all of this into one module. The top level block diagram for our car detection FSM is shown in figure 3 below. It consits of 4 separate block modules that are all under the DE1_SoC top level. Each modules inputs are labeled as arrows going into them from the top, and outputs are labeled as arrows coming out of the blocks at the bottom. The car sensor and counter modules both have outputs that are inputs to other modules and it is shown in figure 3.

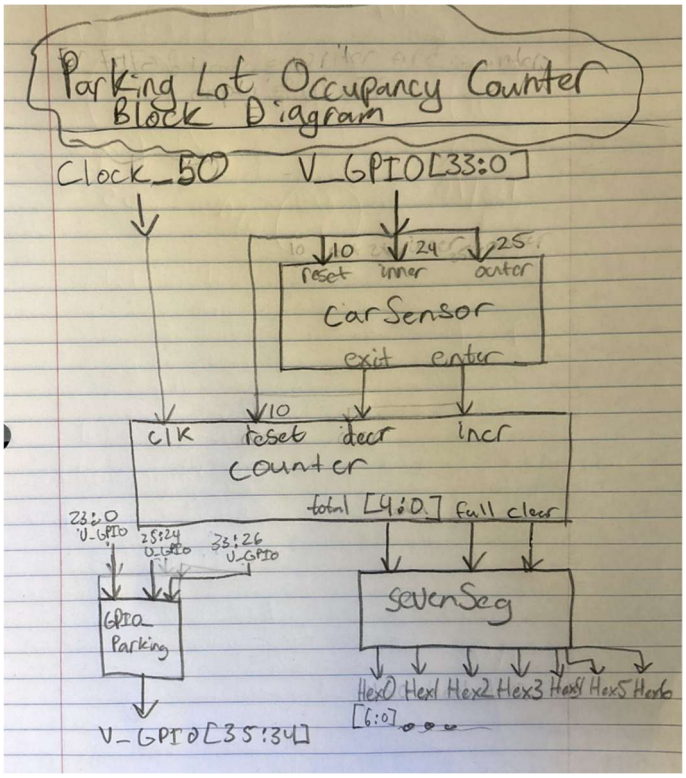

Figure 4: Shows the Top-Level Block Diagram for our Parking Lot organizer

Results

Our system is a parking lot occupancy counter that tracks the number of cars entering and exiting a lot using two photosensor signals. The system determines the direction of movement by monitoring the sequence of the outer and inner sensors and uses a finite state machine (FSM) to generate single-cycle enter and exit signals for valid car movements. These signals are then used to increment or decrement a counter representing the current number of cars in the lot, with a maximum capacity of 18 and a minimum of 0.

GPIO_parking

This module is responsible for interfacing the FPGA with off-board components using the GPIO pins. Specifically, it reads two switch inputs that emulate the parking lot photosensors (outer and inner) and drives two LEDs to reflect their values in real time. An additional GPIO pin is used to read a reset signal. The inputs and outputs are mapped out in Figure 5.

```
// Assign GPIO pins to meaningful names
logic outer, inner, reset;

// Read switches (inputs)
assign outer = V_GPIO[24];  // switch 1
assign inner = V_GPIO[25];  // switch 2
assign reset = V_GPIO[10]; //reset switch

// Drive LEDs (outputs)
assign V_GPIO[35] = outer;  // LED 1
assign V_GPIO[34] = inner;  // LED 2
```

Figure 5: Shows the names and mapping of switches and LEDs

In Figure 6 below, we have shown the model sim of our simulation for GPIO_parking. The most important thing to note is that GPIO[24] corresponds to GPIO[35] and GPIO[25] corresponds to GPIO[34] (outer/inner switch to LED). First, we tested reset, then we tested outer triggering, followed by inner triggering and then both. Looking at 50, 100, 150, and 200ps we can see that we have our specific changes and that it continues to follow the behavior/correlation established beforehand. Overall, the simulation verifies that the GPIO interface correctly reads external inputs and drives outputs, providing a reliable connection between the physical components and the internal system logic.

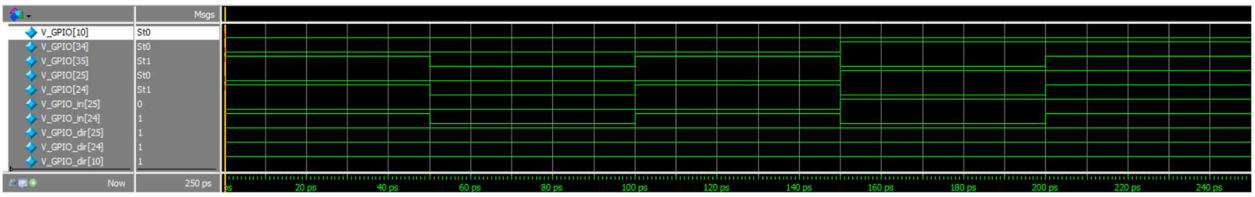

Figure 6: GPIO_parking Model Sim that shows the behavior of pins based on specific inputs

CarSensor Module

In Figure 7 below, we have shown the model sim for our carSensor module. Given two inputs: outer and inner, and two outputs: enter and exit, the state of the inputs determine what is outputted. First, we showed that outer was triggered first, then held while inner is also triggered. Then outer will turn off and inner will stay on for one more clock cycle. The moment outer turns off, the enter signal becomes high which properly represents the logic in our FSM from figure 1. Similarly, at time 2500ps the reverse happens, inner is triggered, then outer is also triggered while inner turns off. As soon as inner turns off, the exit signal becomes high and will stay high for only one clock cycle. This accurately corresponds to our FSM in figure 1. Lastly, we tested to see what would happen if a car enters and stays in its position for a few clock cycles, and in Figure 7, we can see that nothing will happen.

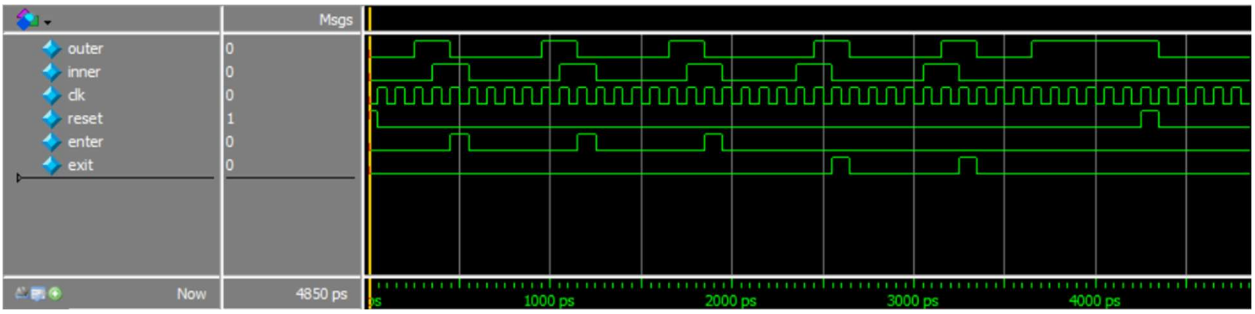

Figure 7: Car sensor Model Sim that shows the outputs given the inputs

Sevenseg

Sevenseg in our module meant to take our specific values and display the correct 7segment display Hex's based on that. Specifically the number of cars from 0-18 and whether the parking lot is full or clear. Looking at Figure 8, the first case we texted was if the system was empty. We can see at 50ps that clear becomes 1 (on) and that the HEXs display the correspond values (an easy way to check is that HEXs 5-2 have values [not normally the case], HEX 1 contains the value for r and not 1 or clear [all 1s], and finally that HEX 0 is display a 0). The next case is at 350ps and that is for full. It displays full (same reasoning as above for HEXs 5-2 however for HEX 1 we can see that it contains the value for 1 and for HEX 0 it contains the value for 8 [so FULL18]). Finally, we tested from zero and added one to test all cases of the outputs. While we cannot see the exact values of HEX 0 on Figure 8, we can see the beginning digit changing accordingly, and at 1650ps we can see that HEX1 changes from 0 to 1 (indicating that we are now at 10). Finally, looking at the last point at 2450ps we can see we get to 18 but don't pass FULL, this is because in our test bench full is still equal to 0. We have logic outside of the Sevenseg module so if the number 18 gets passed for total full would be 1 so this is acceptable.

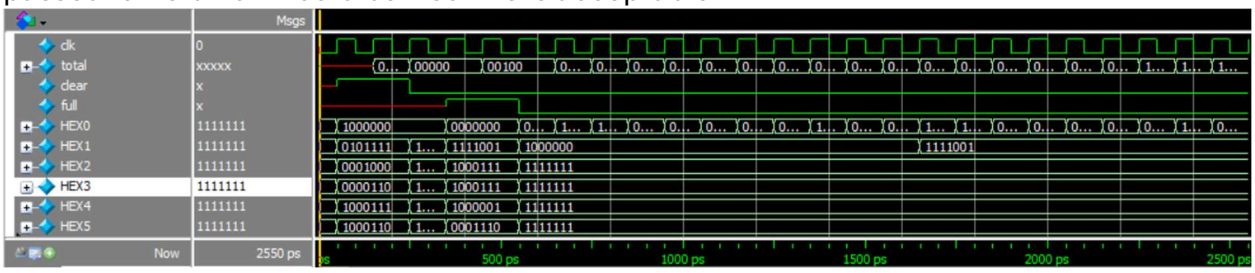

Figure 8: Sevenseg Model Sim showing HEX display for the inputs full, clear, and total.

counter

This module tracks the capacity of the lot, taking in the number of cars that have entered vs. Exited. In the modelsim below, it shows that the cars entering get incremented all the way to 18. Once it hits the 18 car mark, the full signal turns to high. Then the reset is toggled and the clear signal turns high. After, we increment the cars by five and decrement the cars

by 5 to turn the clear signal high once more. This properly demonstrates how the counter module keeps track of cars in the lot.

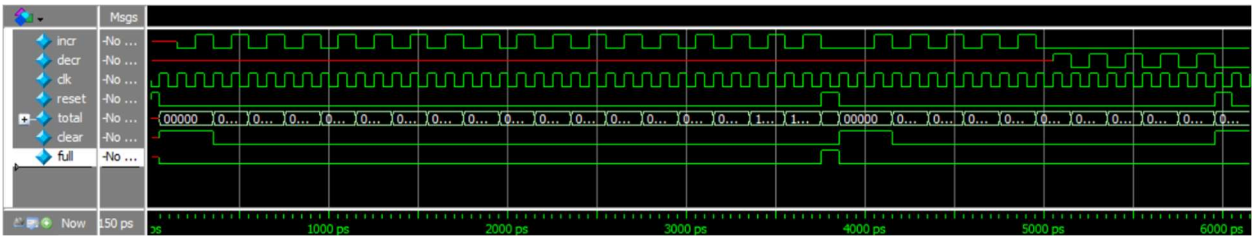

Figure 9: counter Model Sim showing total updating and clear and full responding to total.

DE1_SoC

Figure 10 below illustrates the modelsim for our top-level, DE1_SoC module. The inputs to the system are V_GPIO[24] and V_GPIO[25] which correspond to the outer and inner sensors respectively. First, we tested 20 cars entering, and saw that each time, the HEX displays incremented by one until the lot became full. At that point, the hex display stayed the same for the last two cars. Next, we made all 20 cars exit the lot and saw that the HEX displays decremented by one until it reached the clear display. The HEX displays accurately represent the functionality of our counter and display modules while the enter and exit signals show the functionality of our carSensor module.

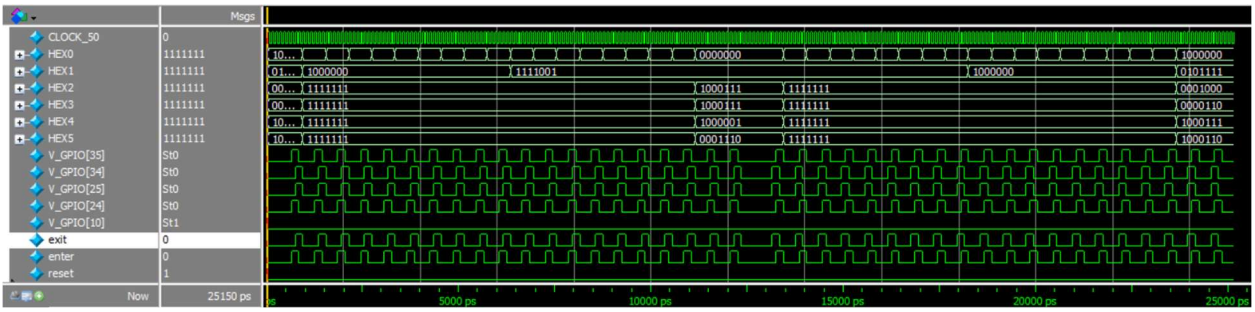

Figure 10: Modelsim for the top-level DE1_SoC module

Flow Summary

Flow Status	Successful - Thu Apr 09 15:41:51 2026
Quartus Prime Version	17.0.0 Build 595 04/25/2017 SJ Lite Edition
Revision Name	DE1_SoC
Top-level Entity Name	DE1_SoC
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	N/A
Total registers	7
Total pins	103
Total virtual pins	0
Total block memory bits	0
Total DSP Blocks	0
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0
Total DLLs	0

Total block memory bits

Figure 11: The ModelSim Flow Summary of compilation of the DE1_SoC module.

Experience Report

We found this lab to be rather straightforward in what was expected and felt that the lab specification did a good job of guiding us through the lab. It took us a while to figure out how exactly the off-board components worked. In addition, we had to iron out our logic for FSM, particularly for the case of a pedestrian walking through the sensor. Once we ironed out the logic building the specific module was not too hard. If we had to change one thing it would be spending more time designing so that coding time was knocked down. We ran into several small logic tweaks we had not realized until we were coding and looking at test benches.

This lab took us approximately 8 hours, broken down as follows:
- Reading – 30 minutes
- Planning – 15 minutes
- Design – 15 minutes
- Coding – 4 hours
- Testing – 1 hour
- Debugging – 2 hour